

NubeUltima

Plataforma Cloud para Telemetria IoT de Ultima Milla

Resumen del proyecto

Curso	Sistemas Cloud y Tecnologias de Ultima Milla (EHP 1)
Catedratico	Sergio Saenz
Repositorio	github.com/Cua17/sistemas-cloud-proyecto (privado)
Hardware	Raspberry Pi 4 (4 GB) + microSD 128 GB

1. Que hicimos

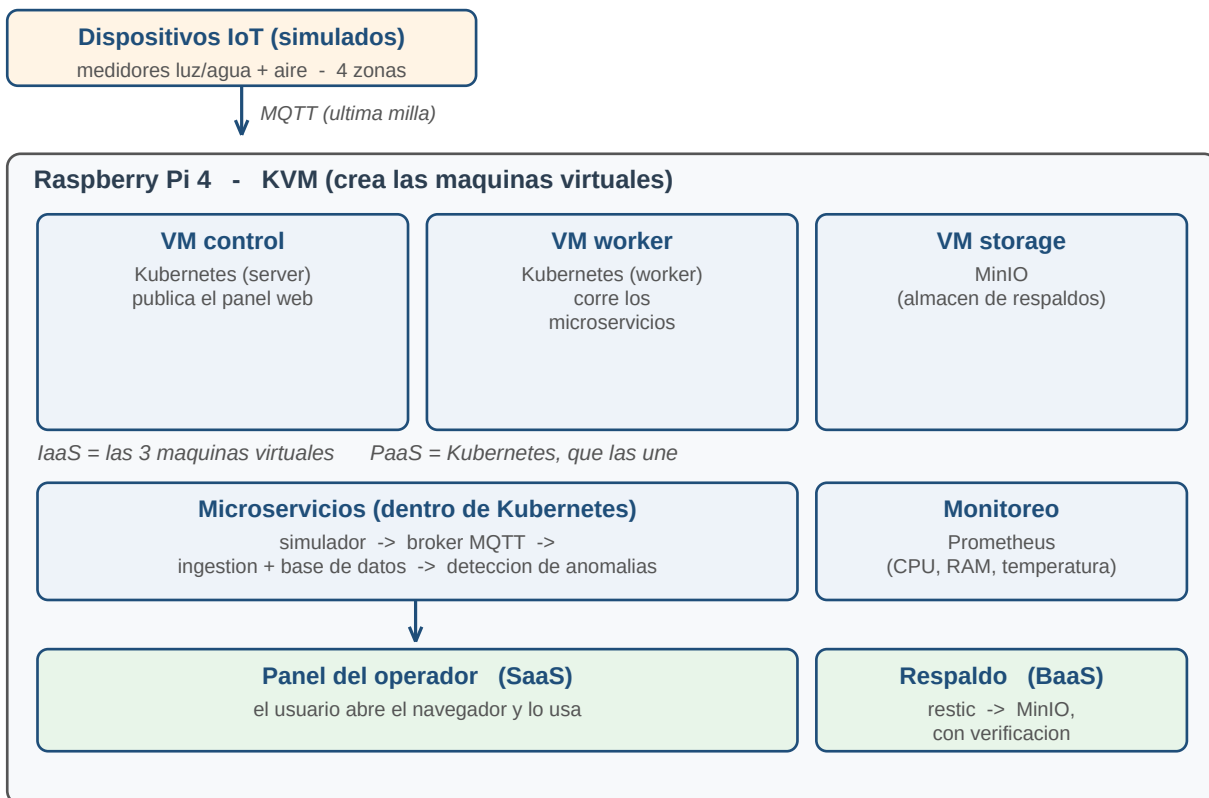
Construimos, dentro de una **Raspberry Pi**, una version en miniatura del sistema en la nube que usaria una empresa de telecomunicaciones para ofrecer un servicio de **ciudad inteligente**.

El escenario: una ciudad con **medidores de luz y agua** y **sensores de calidad de aire** repartidos en 4 zonas. Esos dispositivos (simulados por un programa, porque no tenemos sensores fisicos) envian sus lecturas por la **ultima milla** usando MQTT (el protocolo real de las redes IoT). Las lecturas llegan a un **centro de datos de 3 maquinas virtuales**, donde una plataforma de microservicios las procesa, detecta problemas (fugas, picos de consumo, aire contaminado) y los muestra en un **panel web** para un operador municipal.

El proyecto demuestra las **4 capas de servicio de la nube**: IaaS, PaaS, SaaS y BaaS, mas monitoreo del hardware.

2. El diagrama

Este es el dibujo que va en la diapositiva principal:



3. Las capas, explicadas facil

Capa	Que es (en una frase)	En el proyecto es...	Herramienta
IaaS	Infraestructura como servicio: te dan maquinas / servidores pelados.	Las 3 maquinas virtuales creadas dentro de la Pi, cada una con su IP fija.	KVM (el virtualizador de Linux)
PaaS	Plataforma como servicio: te dan donde "enchufar" tus apps sin administrar servidores.	Un cluster de Kubernetes que reparte los microservicios entre las VMs y los reinicia si se caen.	K3s (Kubernetes ligero)
SaaS	Software como servicio: abris una pagina y la usas, como Gmail.	El panel del operador: mapa de zonas, medidores y alertas en vivo.	App web propia (Python)
BaaS	Backup como servicio: contratar "que alguien me haga las copias".	Respaldos automaticos y cifrados, con una prueba que verifica que se pueden restaurar.	MinIO + restic
Monitoro	Vigilar cuanto consume el hardware.	CPU, RAM, disco y temperatura de la Pi y las 3 VMs, visibles en el panel.	Prometheus

Kubernetes es un "orquestador": vos le decís "quiero esta app corriendo siempre" y el se encarga de mantenerla viva, distribuirla y reiniciarla. Es el tema central de la presentación 3 del curso.

4. Como funciona: el recorrido de un dato

- 1 Un **medidor** (simulado) genera una lectura: por ejemplo "agua, zona 3, 5 L/min".
- 2 La manda por **MQTT** a un servidor de mensajes (Mosquitto) dentro del centro de datos.
- 3 El microservicio de **ingestion** la recibe, la valida y la guarda en una base de datos (SQLite).
- 4 Las **reglas de anomalia** revisan la lectura: si el flujo de agua es alto y sostenido, genera una alerta de "posible fuga".
- 5 El **panel web** consulta la base y muestra las zonas, los medidores y las alertas, actualizandose cada 10 segundos.
- 6 En paralelo: **Prometheus** vigila el hardware y **restic** respalda todo cada 30 minutos.

5. Como lo construimos (las 3 fases)

Fase	Que se hizo
1. IaaS	Preparar la Raspberry Pi, instalar KVM, y crear las 3 maquinas virtuales con Debian, IPs fijas y comunicacion entre ellas. Todo desde archivos de texto (se puede destruir y recrear con un comando).
2. PaaS + SaaS	Instalar el cluster de Kubernetes (K3s) sobre 2 de las VMs. Programar los 3 microservicios en Python, empaquetarlos como contenedores y desplegarlos. Publicar el panel para verlo desde la laptop.
3. Monitoreo + BaaS	Instalar Prometheus para vigilar el hardware. Montar MinIO como almacen de respaldos. Configurar restic para respaldar automaticamente y verificar que las copias se restauran. Reglas de seguridad de red.

Todo el codigo, los scripts y la documentacion estan en GitHub (45 versiones guardadas).

6. Resumen del proyecto en 5 puntos

Sintetizando, el proyecto se explica en **5 puntos**, uno por cada capa del diagrama:

- 1 **KVM** crea 3 maquinas virtuales sobre la Raspberry Pi. Eso es **IaaS**.
- 2 **Kubernetes** une esas maquinas en una plataforma donde se despliegan los servicios sin administrar servidores. Eso es **PaaS**.
- 3 Un **simulador** manda datos de medidores IoT por **MQTT**, el protocolo real de la ultima milla. Un microservicio en Python los recibe, los guarda y detecta anomalias.
- 4 El operador municipal abre un **panel web** y ve las zonas y las alertas en vivo. Eso es **SaaS**.
- 5 **restic** respalda todo a un almacenamiento **MinIO**, con una prueba que verifica la restauracion. Eso es **BaaS**. Y **Prometheus** vigila el hardware.

En resumen: "Demostramos las 4 capas de servicio de la nube -IaaS, PaaS, SaaS y BaaS- sobre una sola Raspberry Pi, con el caso de uso real de un operador de telecomunicaciones: conectividad IoT de ultima milla mas una plataforma de datos para una ciudad inteligente."

7. Herramientas principales (para el informe escrito)

Herramienta	Para que
Raspberry Pi OS (Debian)	Sistema operativo de la Pi
KVM + libvirt	Hipervisor: crea y gestiona las 3 maquinas virtuales
Debian (ARM64)	Sistema operativo de las 3 VMs
Ansible	Configura las VMs de forma automatica y repetible
K3s (Kubernetes)	Orquestador de contenedores (PaaS)
Eclipse Mosquitto	Servidor de mensajes MQTT
Python + FastAPI	Los 3 microservicios propios y el panel
SQLite	Base de datos de la telemetria y las alertas
Prometheus	Monitoreo del hardware
MinIO + restic	Servicio de respaldo (BaaS)
Git + GitHub	Control de versiones del proyecto

8. Cumplimiento de la rubrica

Lo que pide	Como lo cumplimos
Las VMs deben "compilar"	Se crean desde codigo en ~85 segundos con un comando
IPs estaticas en la red local	192.168.100.11 / .12 / .13, fijas
Comunicacion bidireccional estable	0% de perdida en las 6 direcciones, latencia < 1 ms
Los 4 tipos de servicio funcionales	IaaS, PaaS, SaaS y BaaS, los 4 corriendo
Monitorizar el HW utilizado	Prometheus mide CPU/RAM/disco/temperatura de las 4 maquinas
Acceso exitoso a los servicios	El panel se abre desde la laptop; demo en vivo
Verificacion de los respaldos	Prueba automatica que restaura y valida la base de datos
BaaS (puntos extra)	MinIO + restic con respaldos programados y verificables